A timetable that respects people

Automatic scheduling for 626 students, 54 teachers and 63 rooms — fewer footsteps, no stranded periods, every rule independently proven, and a plan for when something breaks.

−56% student walking per week

−93% idle periods between classes

0 violations of 10 hard rules, certified independently

10/10 planted faults caught by the checker

A The problem and the school

626 students, 54 teachers, 63 rooms, 33 periods a week

THE BRIEF · Question.txt, verbatim

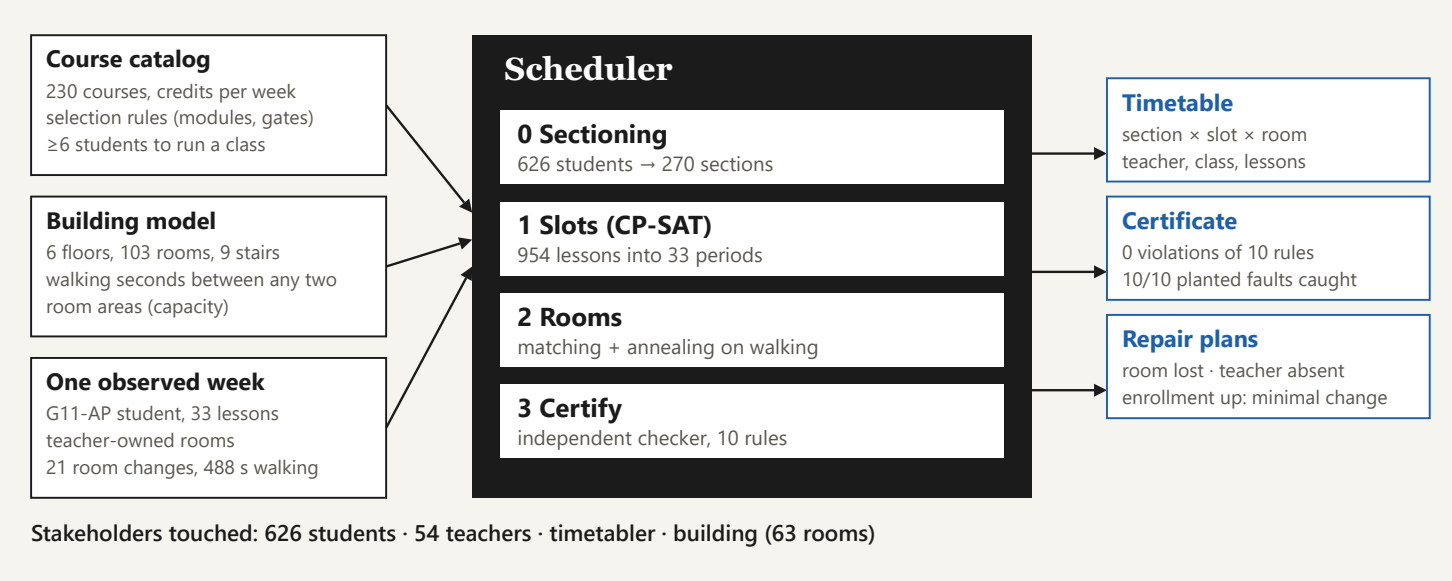
The following is a teacher's hackathon activity based on a hypothetical high school scenario.

The "school" has to produce a timetable that takes care of teachers, students, classrooms and courses all at once. This timetable affects the daily study and work of teachers and students, so having a high-quality timetable is important. We want you to consider the constraints in the school's timetable and optimize the timetable. By this, you should design a system and be able to explain why it is better than what we currently have. An example of a constraint is class hours: you may change Math class with English class, but the hours spent on math class in a semester should not change. The result should be a system that automatically schedules timetables. There is no standard answer. How the current system works, which constraints must be satisfied, which aspects are worth improving: all of these are for you to research, judge and trade off, and you should explain your reasoning in your submission. During the presentation, the judges will ask why you made these decisions.

2. Evaluating Your Results This problem cares more about which aspects you think can be improved and how you improve them. Your submission should answer:

- What problems in the current timetable system did you identify?
- Which aspects did you choose to optimize (for example, minimizing student movement between classrooms)? How did you optimize them?
- Who do these aspects affect, and how much? Why do they really matter?
- Compared with the current timetable, how much does your solution improve things? How do you measure it?
- How do you confirm that your timetable satisfies every constraint it must satisfy?
- Can your system still cope when circumstances change? How to present your results most convincingly is also up to you.

Q1	Problems in the current system → **B**
Q2	Aspects chosen and how they are optimised → **C, D**
Q3	Who is affected, how much, why it matters → **D**
Q4	Improvement vs. today and how it is measured → **D**
Q5	Proof that every constraint holds → **E**
Q6	Coping when circumstances change → **F, G**



Course catalog
230 courses, credits per week
selection rules (modules, gates)
≥6 students to run a class

Building model
6 floors, 103 rooms, 9 stairs
walking seconds between any two
room areas (capacity)

One observed week
G11-AP student, 33 lessons
teacher-owned rooms
21 room changes, 488 s walking

Scheduler
0 Sectioning 626 students → 270 sections
1 Slots (CP-SAT) 954 lessons into 33 periods
2 Rooms matching + annealing on walking
3 Certify independent checker, 10 rules

Timetable
section × slot × room
teacher, class, lessons

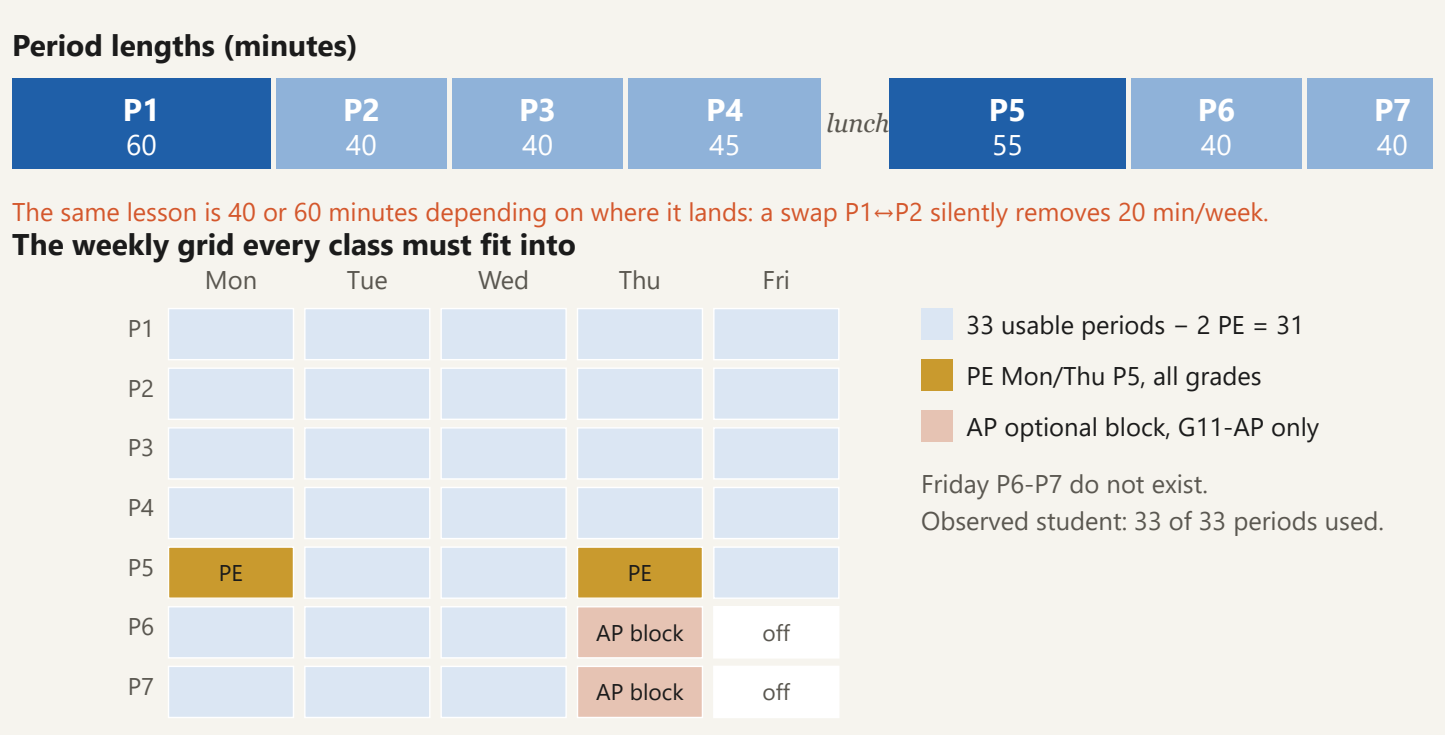
Certificate
0 violations of 10 rules
10/10 planted faults caught

Repair plans
room lost · teacher absent
enrollment up: minimal change

Stakeholders touched: 626 students · 54 teachers · timetabler · building (63 rooms)

Fig. 1 — *What goes in, what the system does, what comes out. The only real timetable data is one observed week; everything else is simulated from the catalog and the building model.*

Period lengths (minutes)



The same lesson is 40 or 60 minutes depending on where it lands: a swap P1↔P2 silently removes 20 min/week.

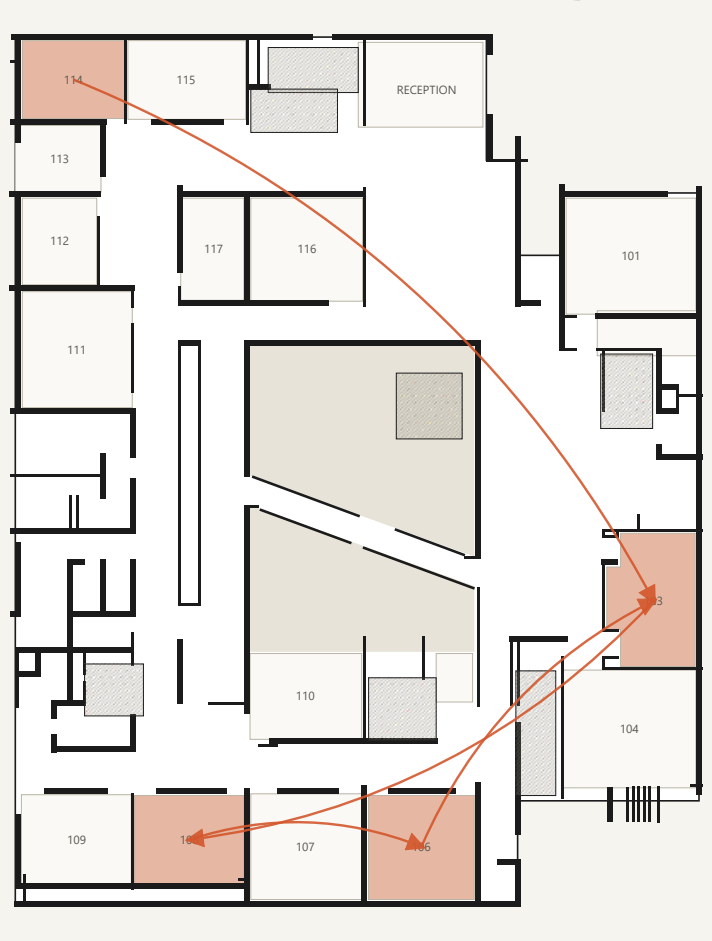
The weekly grid every class must fit into

Friday P6-P7 do not exist.
Observed student: 33 of 33 periods used.

Fig. 2 — *Periods are unequal and the grid has fixed holes. Contact hours must be counted in minutes, not lessons.*

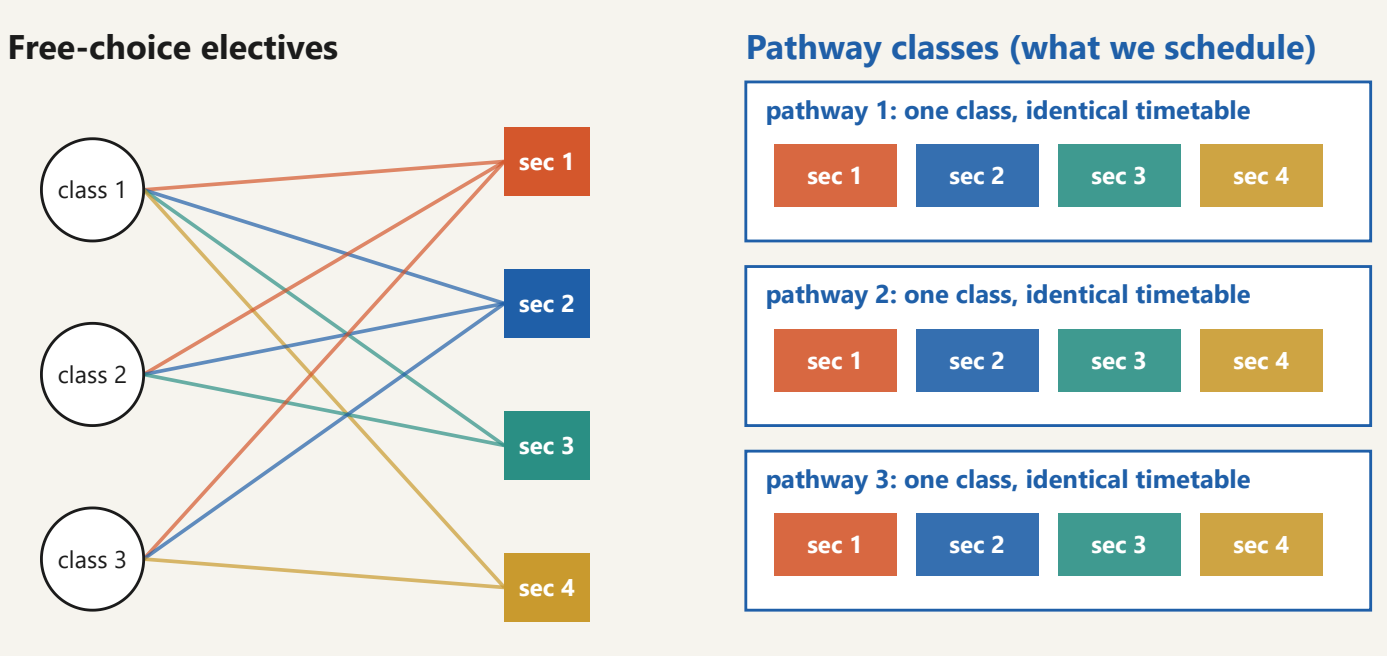
B What is wrong today

Q1 — problems in the current timetable system



Monday of the one real week: 114 → 103 → 108 → 108 → 106 → 106 → 103
4 room changes in one day; **21** per week, 488 s of walking.

Fig. 3 — *One G11-AP student's observed Monday on the 1F plan. Every teacher owns a room, so the student walks to the teacher all day.*



Free-choice electives
class 1
class 2
class 3
sec 1
sec 2
sec 3
sec 4
Pathway classes (what we schedule)
pathway 1: one class, identical timetable
sec 1
sec 2
sec 3
sec 4
pathway 2: one class, identical timetable
sec 1
sec 2
sec 3
sec 4
pathway 3: one class, identical timetable
sec 1
sec 2
sec 3
sec 4

Sections tie classes together; lessons that may not overlap
form a clique of 31 > 29 free periods: no valid grid.

Each section holds one class: cliques stay below
the slot count; conflicts shrink to teachers, rooms.

Fig. 4 — *Why the obvious fix does not work: free elective choice couples classes into a clique larger than the free periods. Pathway classes remove the coupling.*

The hours rule nobody checks

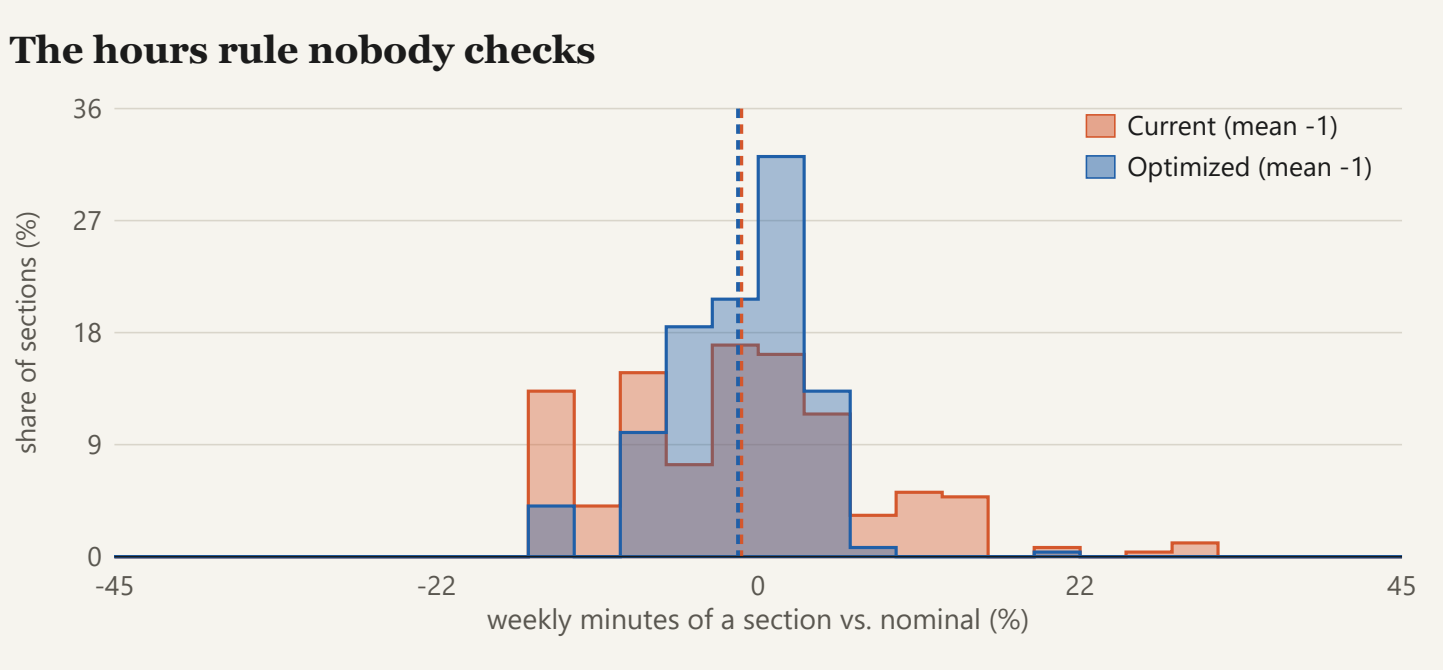
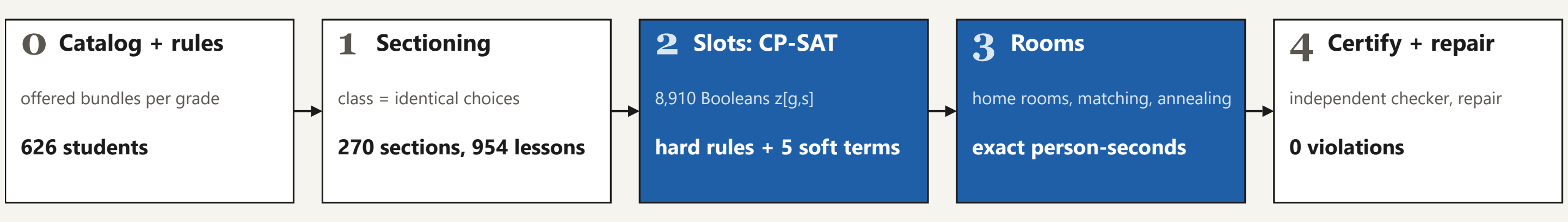


Fig. 5 — *Weekly minutes of every section against its nominal. A grid built without looking at period length puts* **110 of 270** *sections out of the ±8% band; ours puts none (the brief: hours should not change).*

- No objective on students' walking, idle gaps or day shape.
- Repair after a room loss is manual.
- No independent proof that the grid is legal.

C Method

Q2 — what we optimise, and how



0 Catalog + rules
offered bundles per grade
626 students

1 Sectioning
class = identical choices
270 sections, 954 lessons

2 Slots: CP-SAT
8,910 Booleans z[g,s]
hard rules + 5 soft terms

3 Rooms
home rooms, matching, annealing
exact person-seconds

4 Certify + repair
independent checker, repair
0 violations

Fig. 6 — *Four stages, each checkable on its own. Stage 1 uses one Boolean per section and slot, so the model grows linearly instead of with every pair of lessons.*

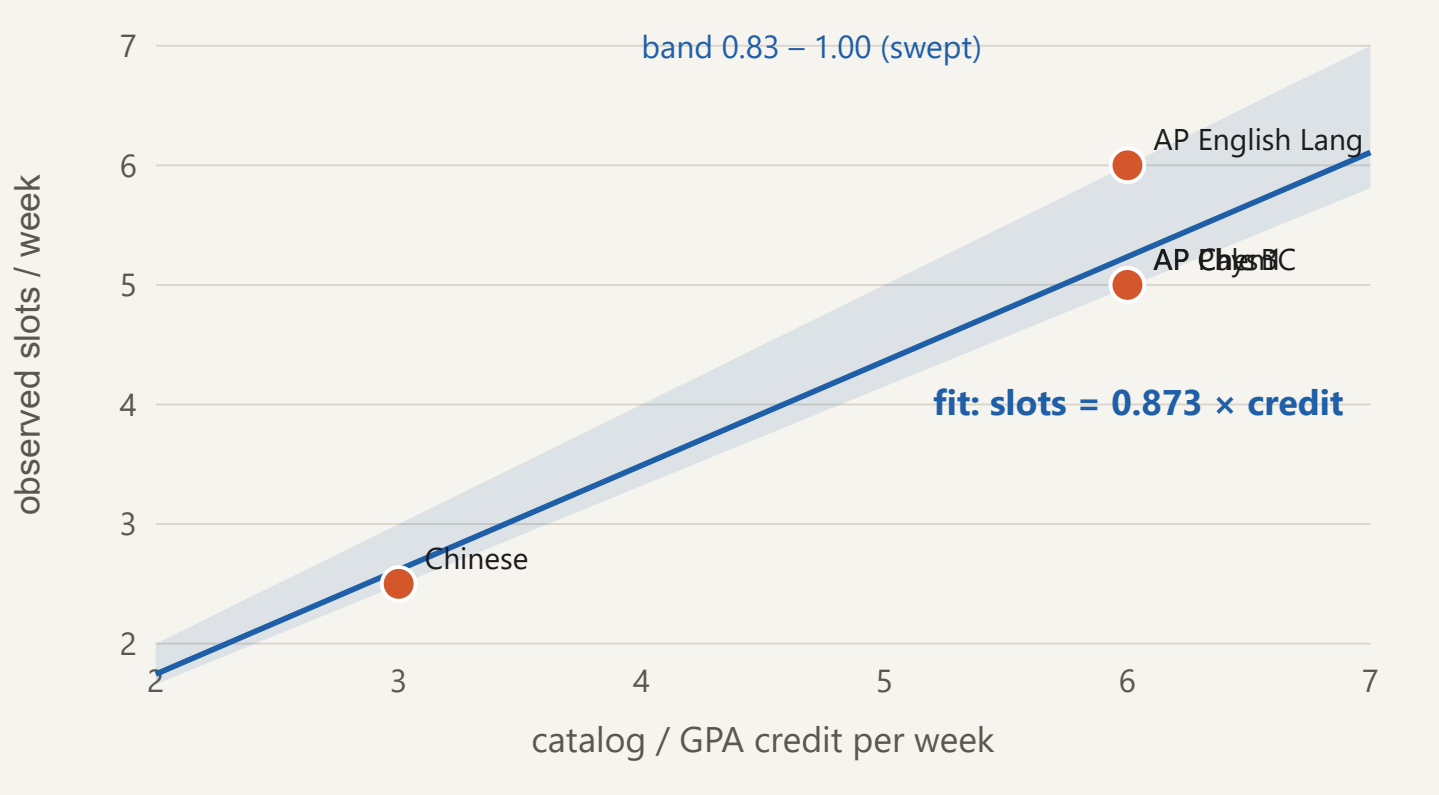


Fig. 7 — *Loads come from the catalog credits through a factor fitted on the observed week (credit → slots), carried as a band, never as a point.*

CREDIT TO WEEKLY SLOTS

$$n_c = \left\lfloor k\kappa_c + \tfrac{1}{2} \right\rfloor, \quad k = \frac{\sum_i \kappa_i o_i}{\sum_i \kappa_i^2} \approx 0.873$$

HARD RULES ($z_{G,S} \in \{0,1\}$)

$$\sum_s z_{g,s} = n_g, \quad \sum_{g\in\sigma} z_{g,s} \le 1, \quad \sum_{g\in T_t} z_{g,s} \le 1$$

$$\sum_{g:\, a_g \ge a} z_{g,s} \le R_\kappa(a) \quad \text{(room area)}$$

CONTACT HOURS IN MINUTES

$$\Big|\sum_s \mu_s z_{g,s} - n_g\bar\mu\Big| \le \max(\tau n_g \bar\mu,\, 9),\ \tau = 0.08$$

STAGE 1: SLOT OBJECTIVE

$$\min\ w_{gap}\sum_\sigma c_\sigma \Gamma_\sigma + w_h \sum_\sigma c_\sigma H_\sigma + w_{sp}\sum_g |g|\,\mathrm{rep}_g + w_m \sum_{g\ \text{hard}} |g| \sum_s (p_s - 1) z_{g,s} + w_{tg}\sum_t \Gamma_t$$

STAGE 2: PERSON-SECONDS

$$\min_r \sum_{(a,b)\in\mathcal{L}_{stud}} n_{ab}\,\tau(r_a,r_b) + \lambda \sum_{(a,b)\in\mathcal{L}_{teach}} \tau(r_a,r_b)$$

SCORE, FAIRNESS, ROBUSTNESS

$$J = \bar\ell_{stud} + \alpha \bar L_t + \beta\,\mathrm{CVaR}_q(L), \qquad \mathbb{E}[\Delta] = \sum_\xi p_\xi \Delta(\xi)$$

Students stay, teachers walk: λ prices a teacher-second against a student-second (swept in F). Γ = idle gaps, H = days with six classes or more, rep = same-day repeats of one section.

D Results against the current system

Q3, Q4 — who gains, how much, how measured

🟥 Current system (emulator calibrated to the observed week, 24 runs; the one shown is run 0) 🟦 Optimized

Where students change rooms

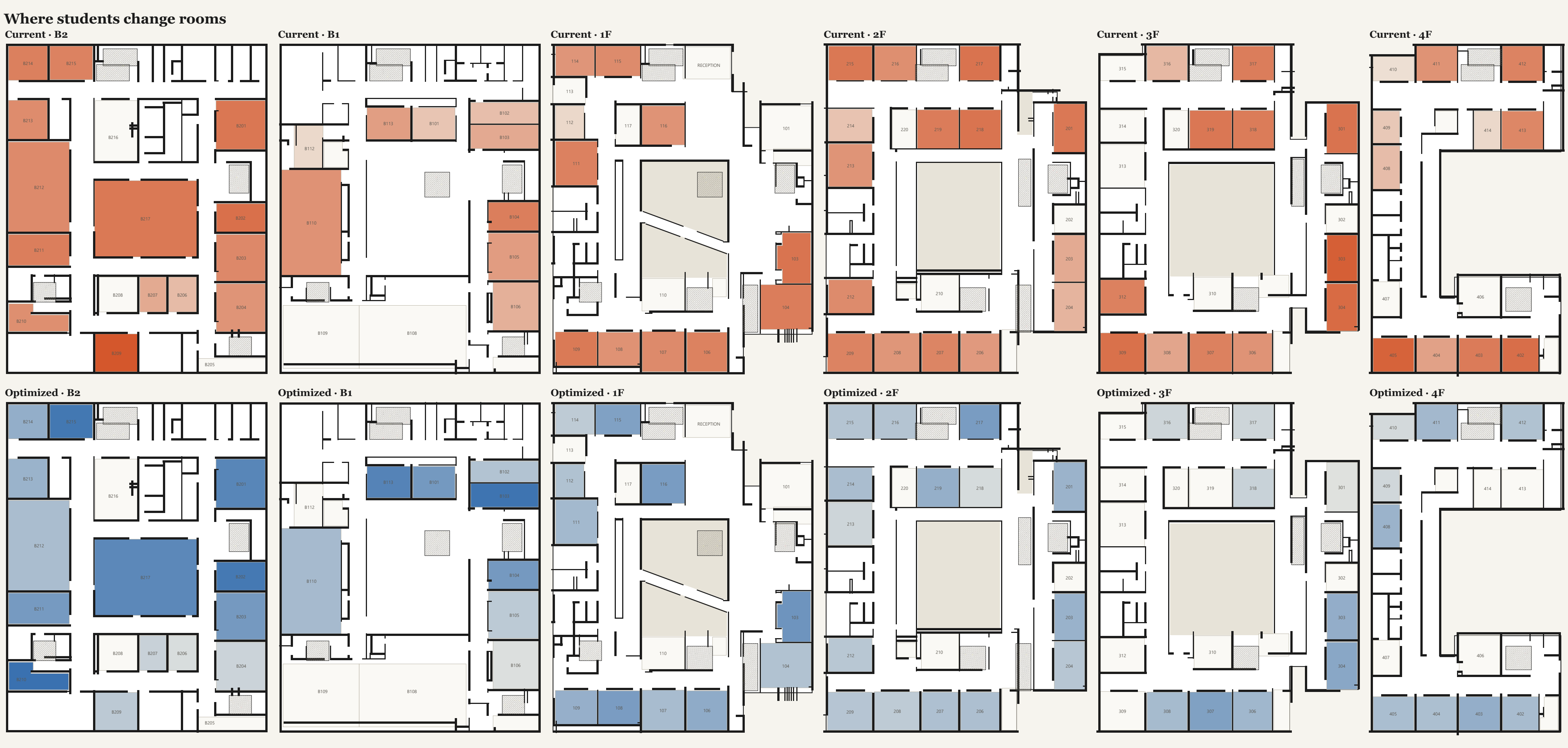


Fig. 8 — *Real building model, six floors. Area of colour = room changes touching that room per week, one common scale. Teacher-owned rooms scatter traffic across the whole building; home-class rooms concentrate it where classes live.*

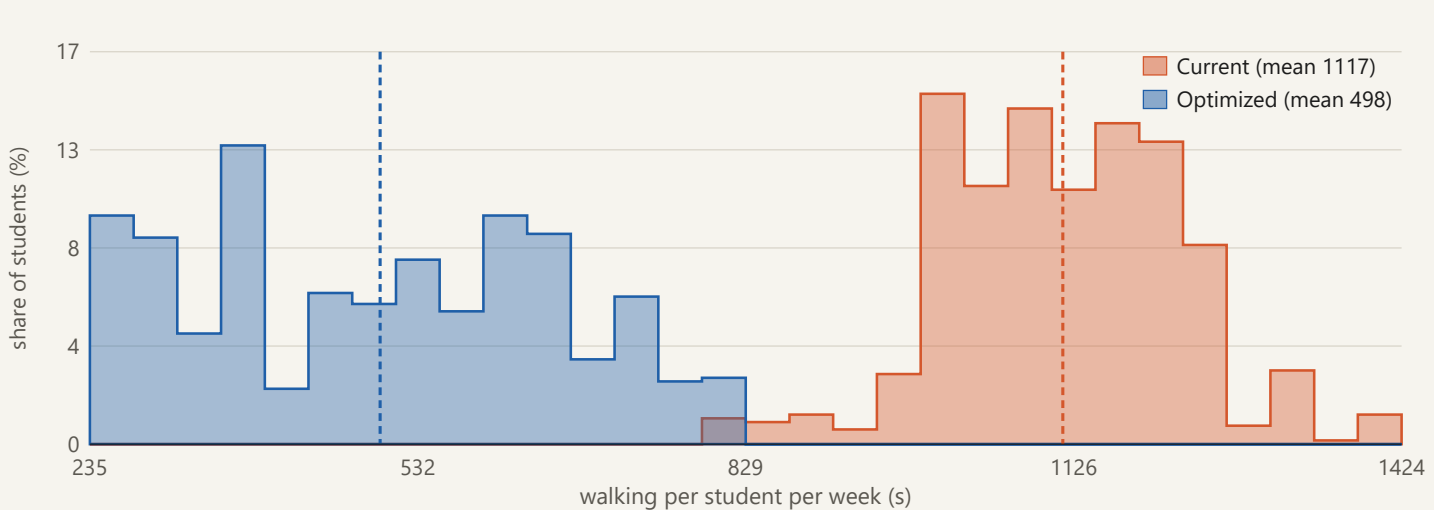


Fig. 9 — *Walking time of every student. Mean 1122 → 498 s (−56%); the worst-20% tail falls too, so the gain is not bought from a few.*

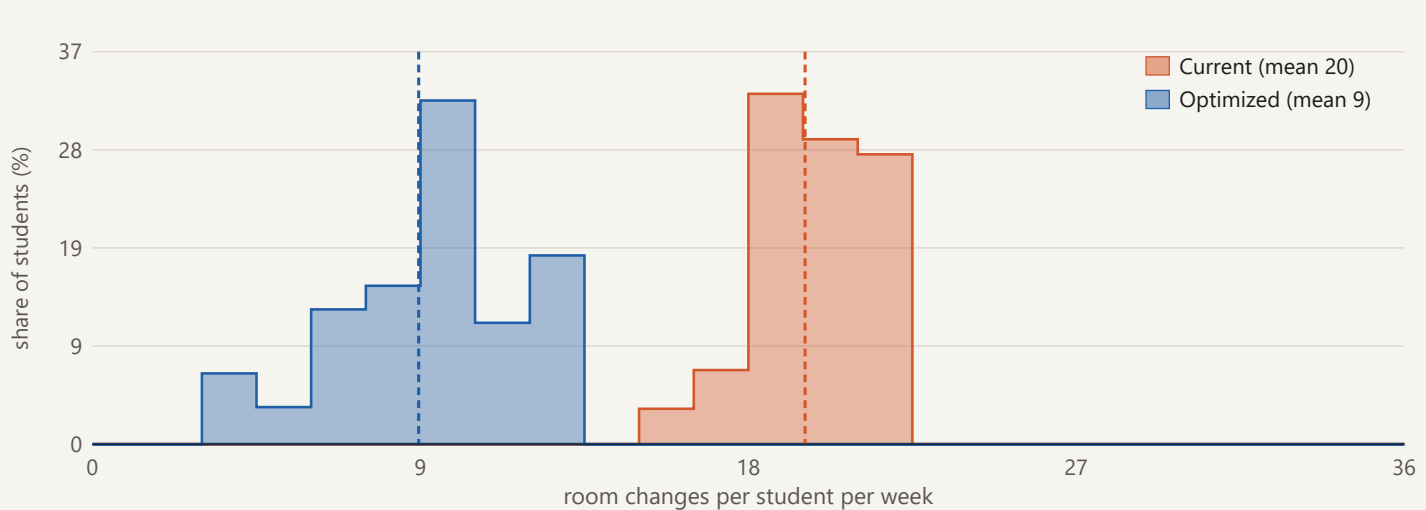


Fig. 10 — *Room changes per student per week (observed week: 21).*

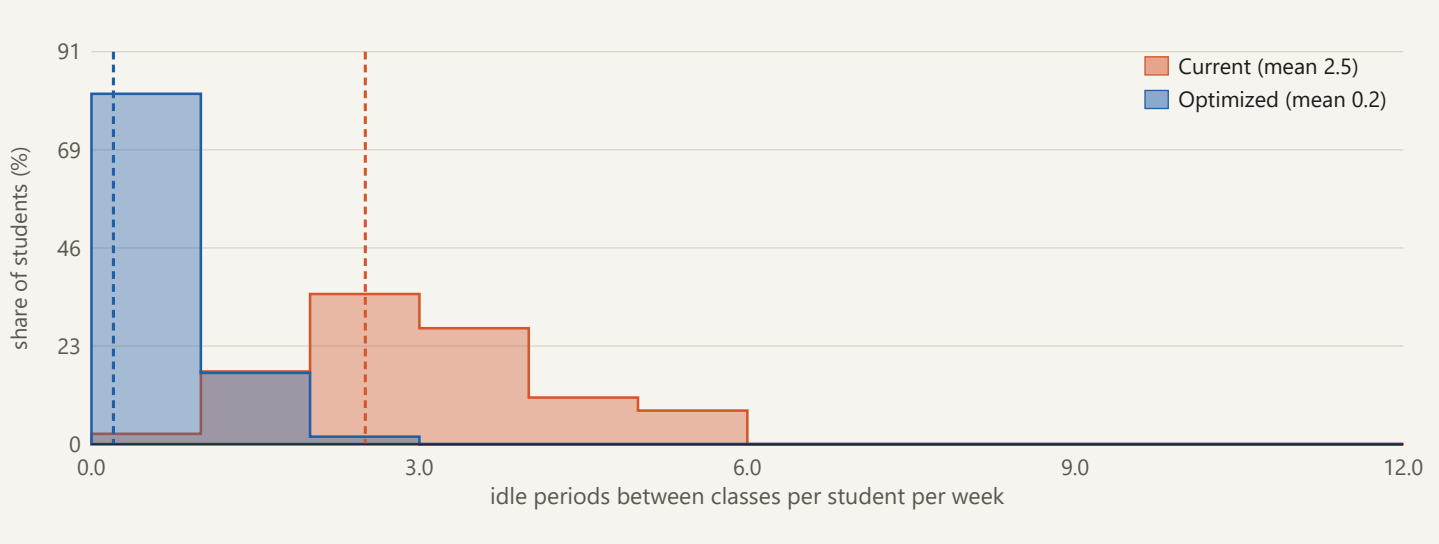


Fig. 11 — *Idle periods stranded between classes, per student per week: −93%.*

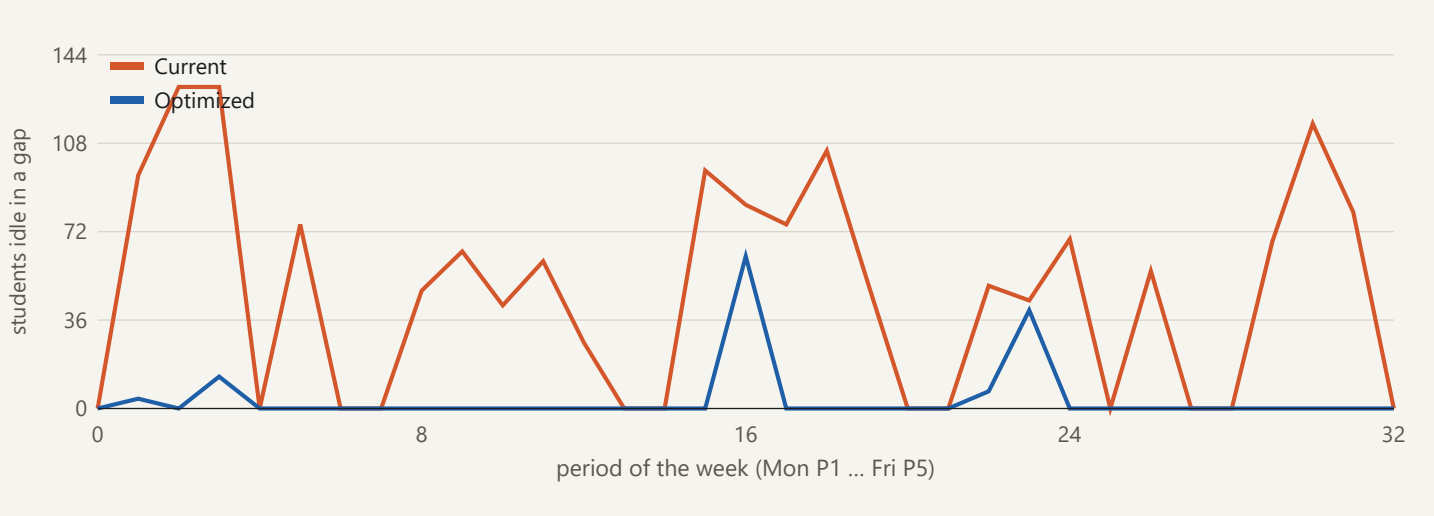


Fig. 12 — *Students sitting in a gap, by period of the week. Optimized grids push free periods to the edges of the day.*

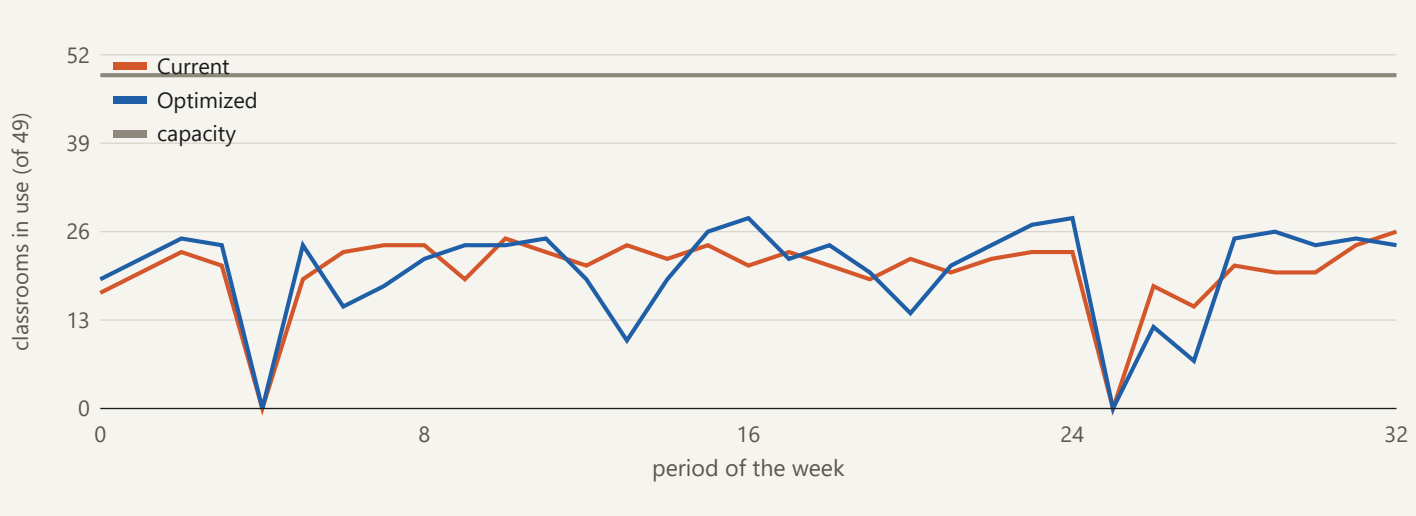


Fig. 13 — *Classrooms in use by period. Demand never nears the 49 available; the building has room to spare, so the saving is not a capacity artefact.*

Which decision buys the saving?

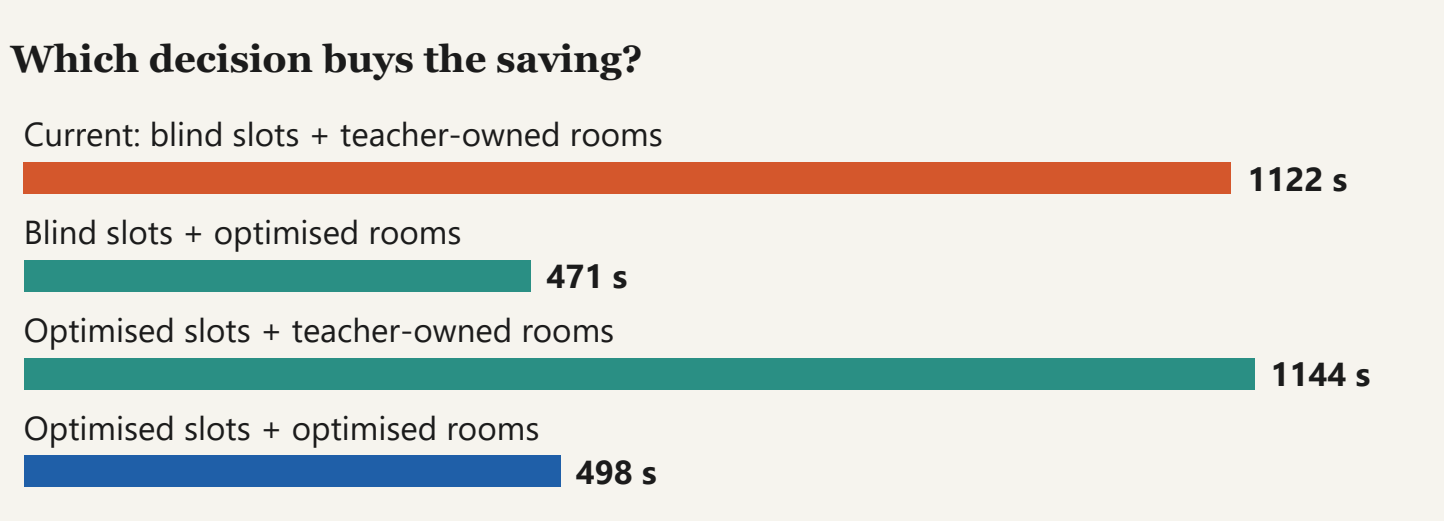
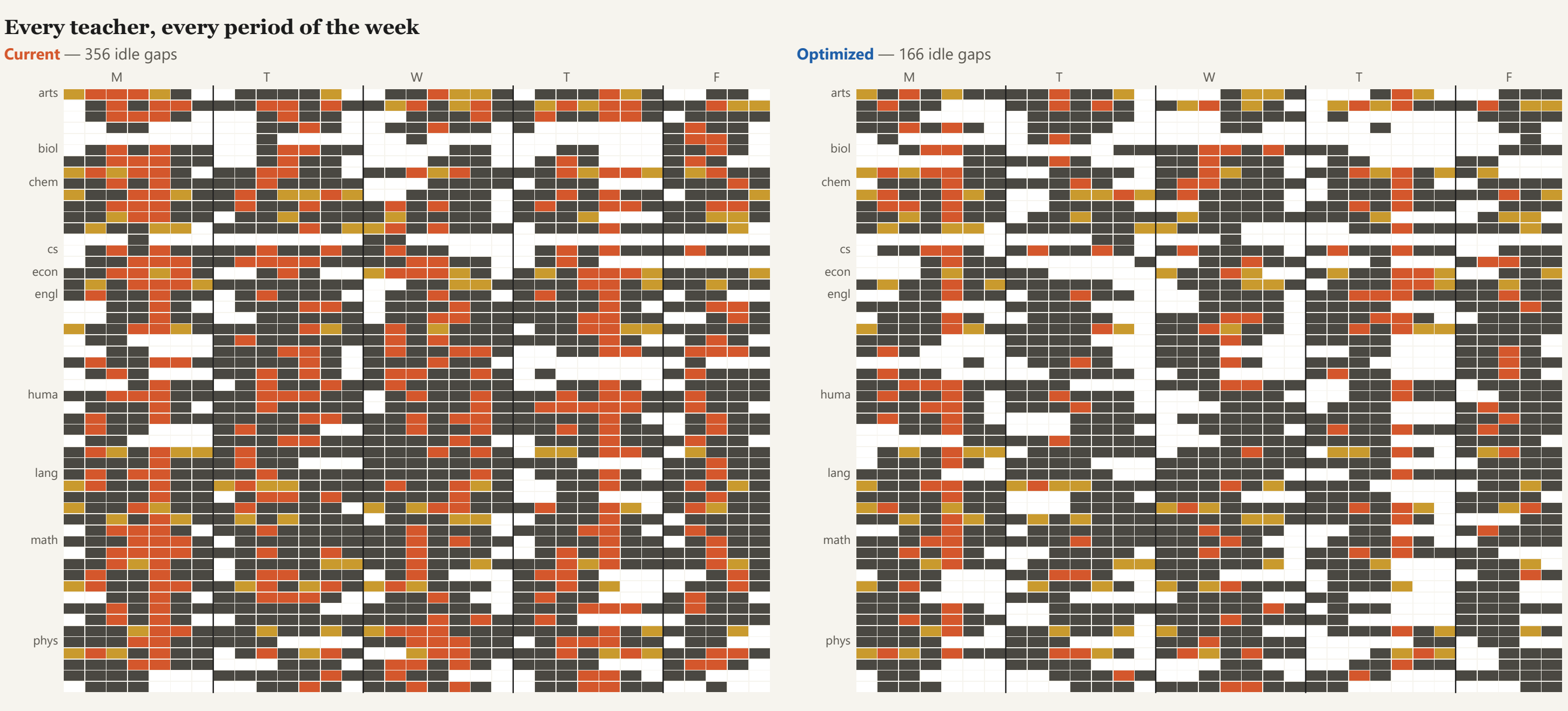


Fig. 14 — *2×2 ablation. Room policy removes the walking; slot policy removes the idle gaps.*

Every teacher, every period of the week



■ teaching ■ busy elsewhere (IB / G12, other building) ■ idle gap between lessons

Fig. 15 — *54 teachers × 33 periods, grouped by department. Orange cells are free periods trapped between two lessons of the same day.*

Who gains, by how much

WHO	MEASURE (PER WEEK)	CURRENT	OPTIMIZED
626 students	walking (s)	1122	**498**
	room changes	19.9	**9.0**
	idle periods	2.77	**0.20**
	worst-20% walking (s)	1301	**724**
	hard subjects: mean period	3.74	**3.27**
54 teachers	idle gaps	6.7	**3.1**
	walking (s)	375	**439**
	schedule loss L	10.8	**10.3**
Everyone	sections off the minute band	110	**0**

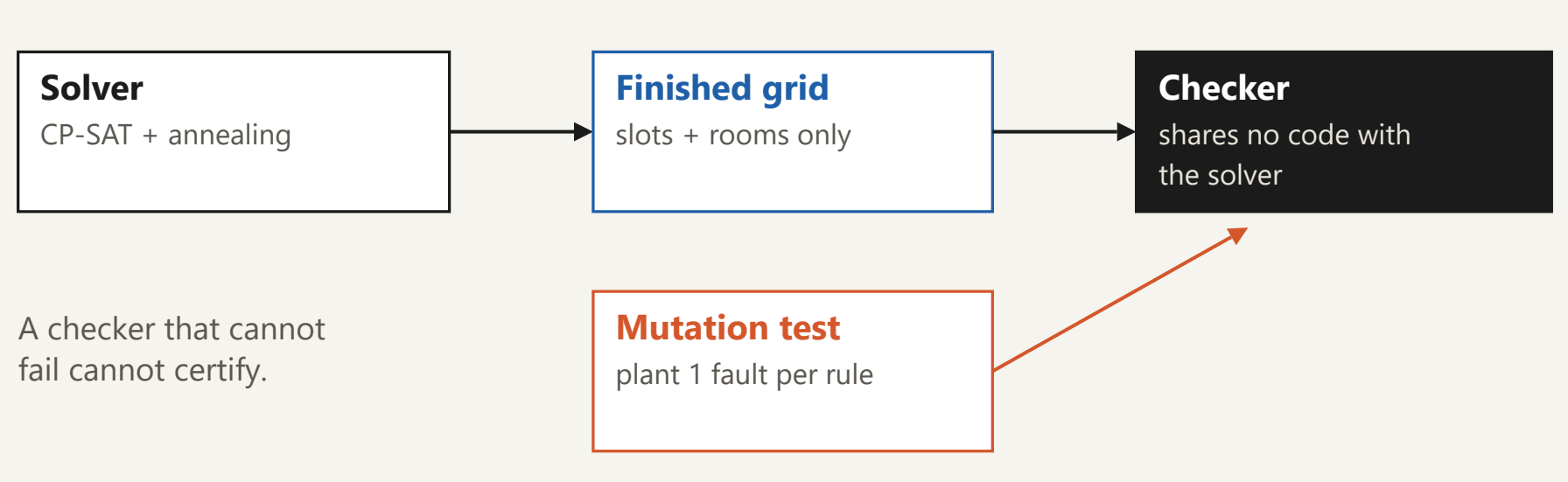
Teachers walk more, deliberately: 626 students stay put and 54 teachers carry the move.

E Proof that every rule holds

Q5 — how we confirm the timetable satisfies every constraint

RULE	HARD CONSTRAINT	OURS	CURRENT	PLANTED FAULT
H1	section meets exactly its weekly slot count, on distinct slots	0	0	caught
H2	slot exists, is not a fixed block for the grade, not teacher-external-busy	0	0	caught
H3	no student in two places at once	0	0	caught
H4	no teacher double-booked; teacher qualified for the department	0	0	caught
H5	room exists, right kind, big enough, not double-booked	0	0	caught
H6	teacher consecutive-period cap	0	0	caught
H7	lab sections have a double period	0	0	caught
H8	weekly minutes within tolerance of nominal (contact-hours invariant)	0	110	caught
H9	each student receives every course's stated weekly slots	0	0	caught
H10	every consecutive-period room change reachable inside the break	0	0	caught

Fig. 16 — *270 sections, 626 students, 54 teachers, 723 room legs checked. Current = mean over 24 emulated grids.*



Solver
CP-SAT + annealing

Finished grid
slots + rooms only

Checker
shares no code with the solver

A checker that cannot fail cannot certify.

Mutation test
plant 1 fault per rule

Fig. 17 — *The checker re-derives every rule from the finished grid. The mutation suite proves it can fail: 10 of 10 planted faults were detected. 20 automated tests cover loads, selection rules, the solver, repairs and a regression on the observed week.*

F Sensitivity: do the conclusions survive?

Q6 — assumptions, structure and capacity, varied one at a time and all at once

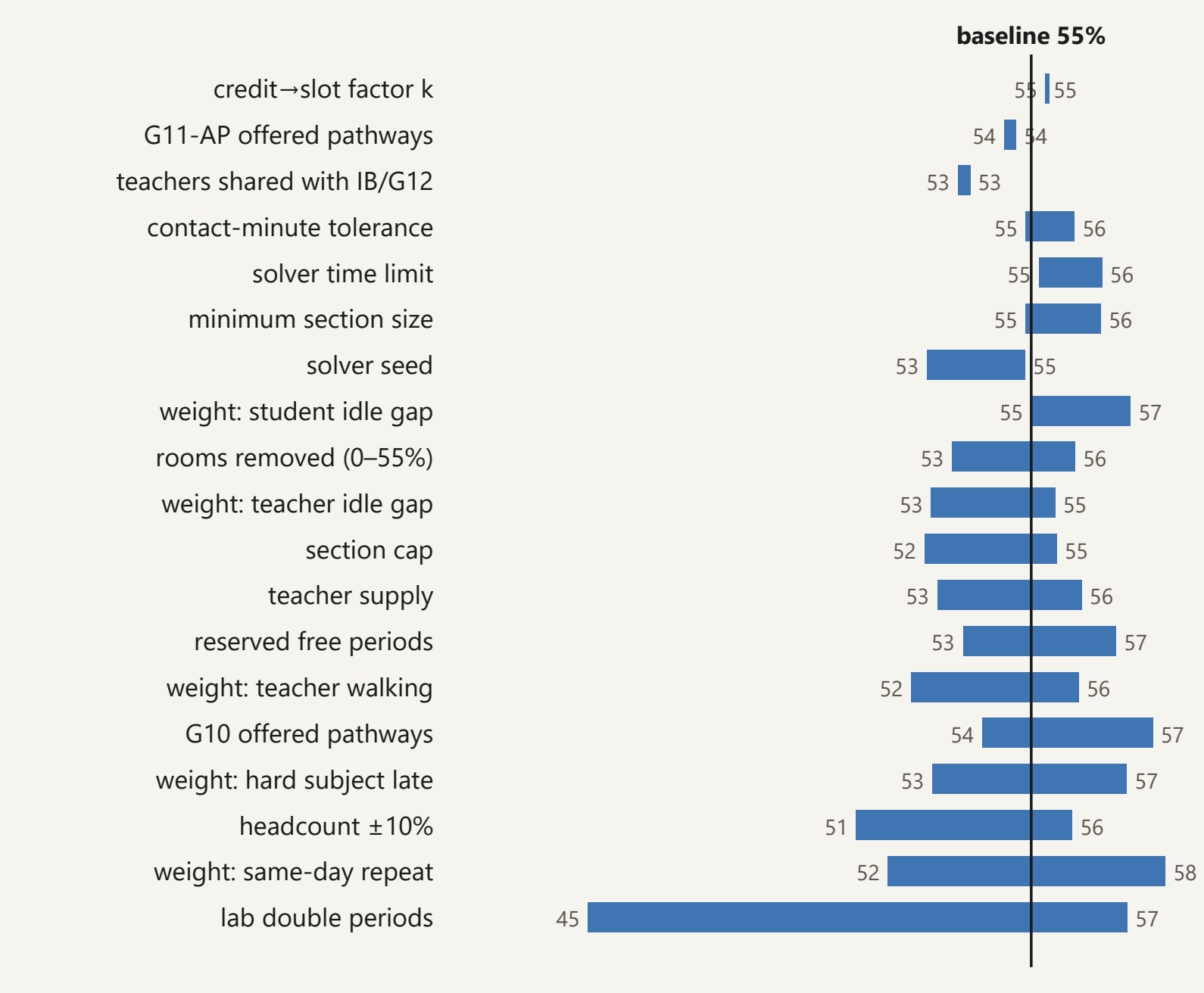


Fig. 18 — *Tornado on the headline: how far each assumption moves the walking reduction vs. the current method (%), 61 runs, each re-solved and re-checked (56 with zero violations).*

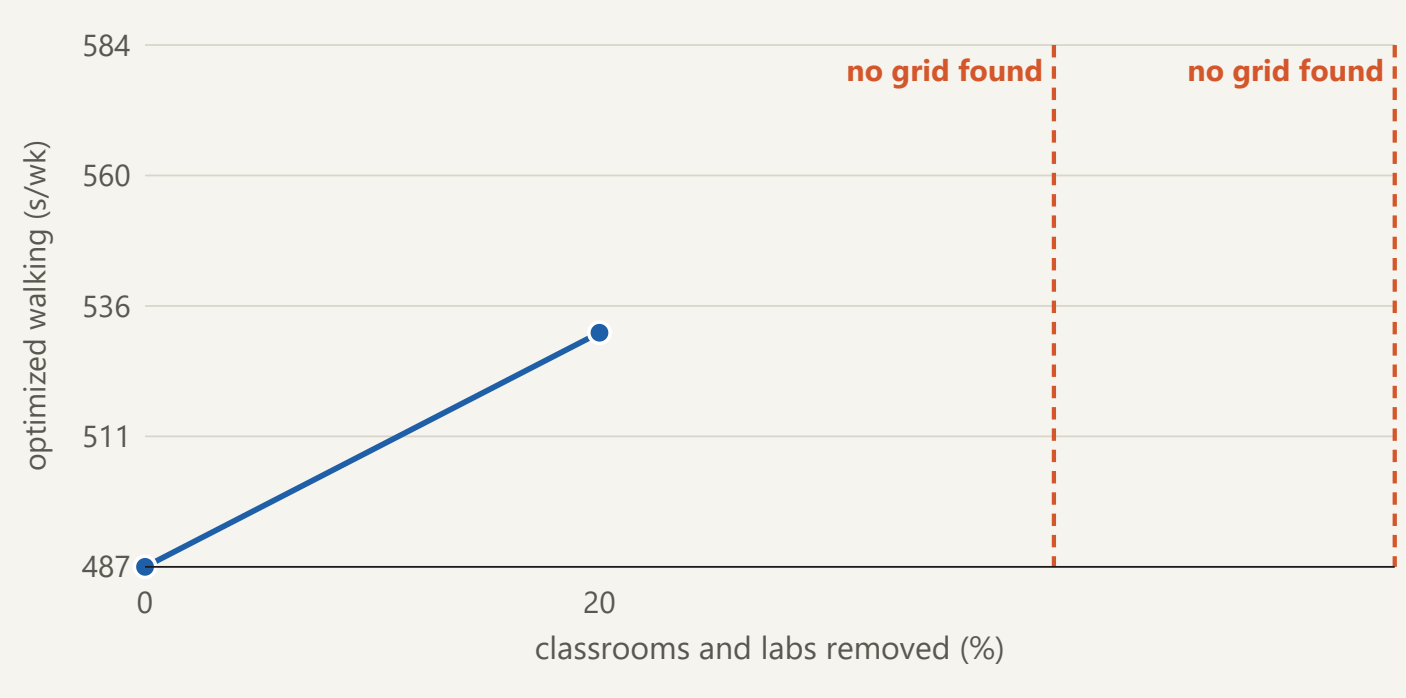


Fig. 19 — *Capacity cliff: between 20% and 40% of rooms removed the instance stops having a valid grid; the system reports infeasible instead of a bad timetable.*

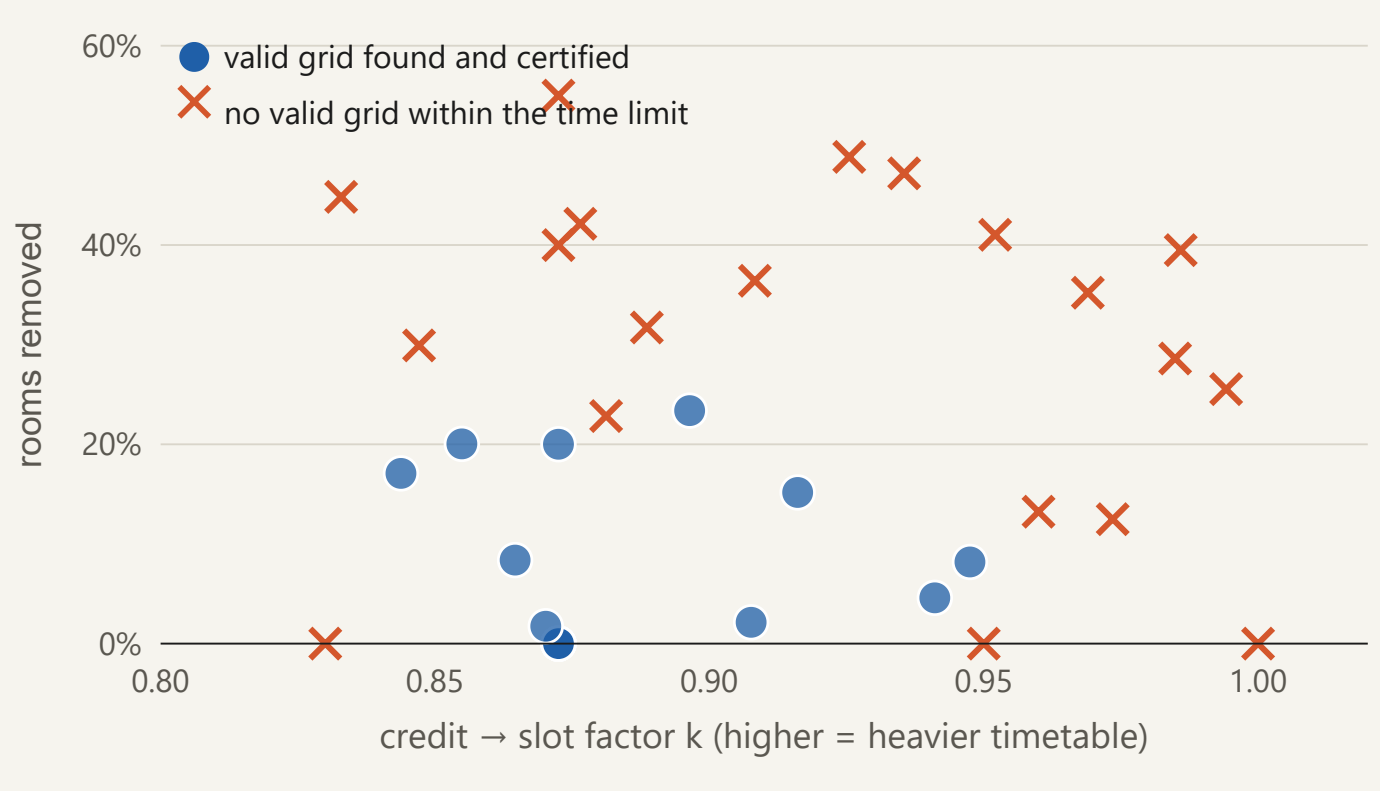


Fig. 20 — *Operating envelope from every sweep run: the load factor k and room loss are the binding assumptions. At k ≥ 0.95 neither the current-style method nor ours found a valid grid in 45–120 s.*

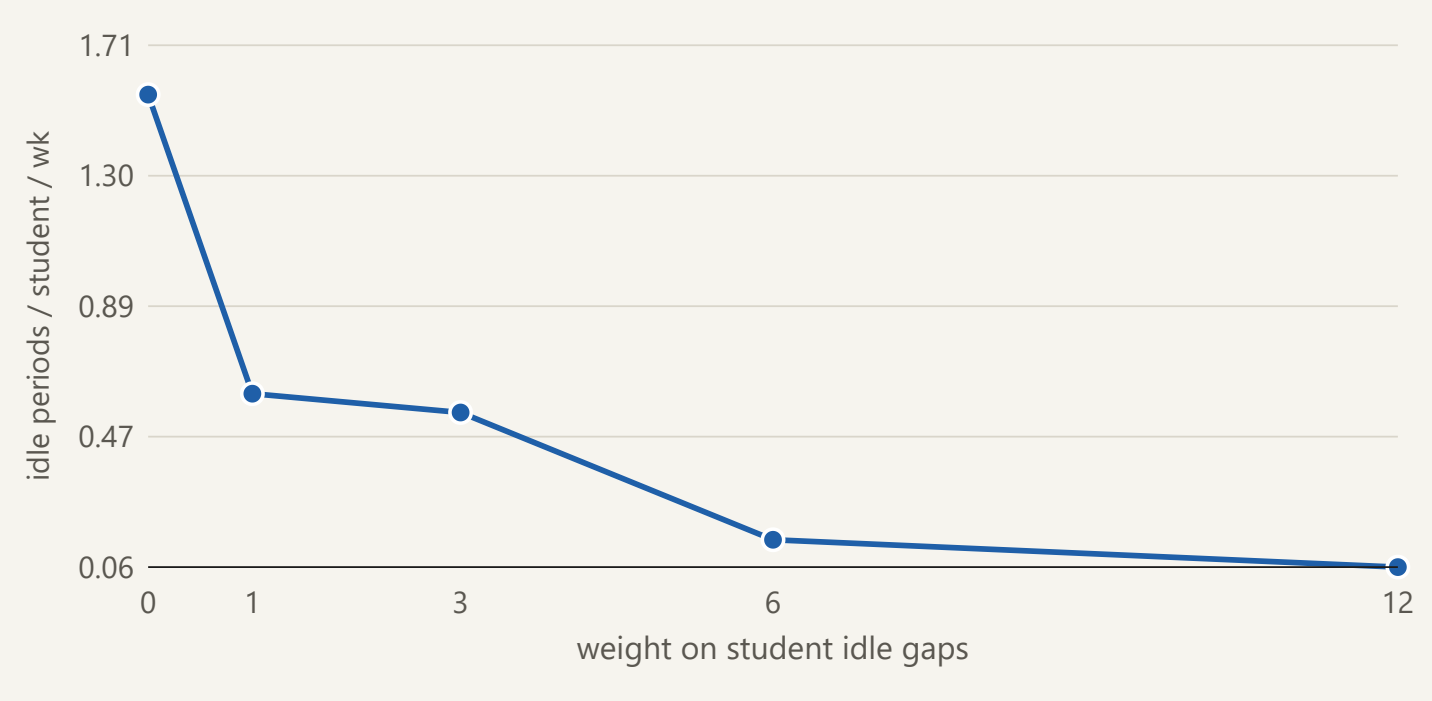


Fig. 21 — *Objective weight vs. outcome: the gap weight buys idle periods smoothly, no cliff.*

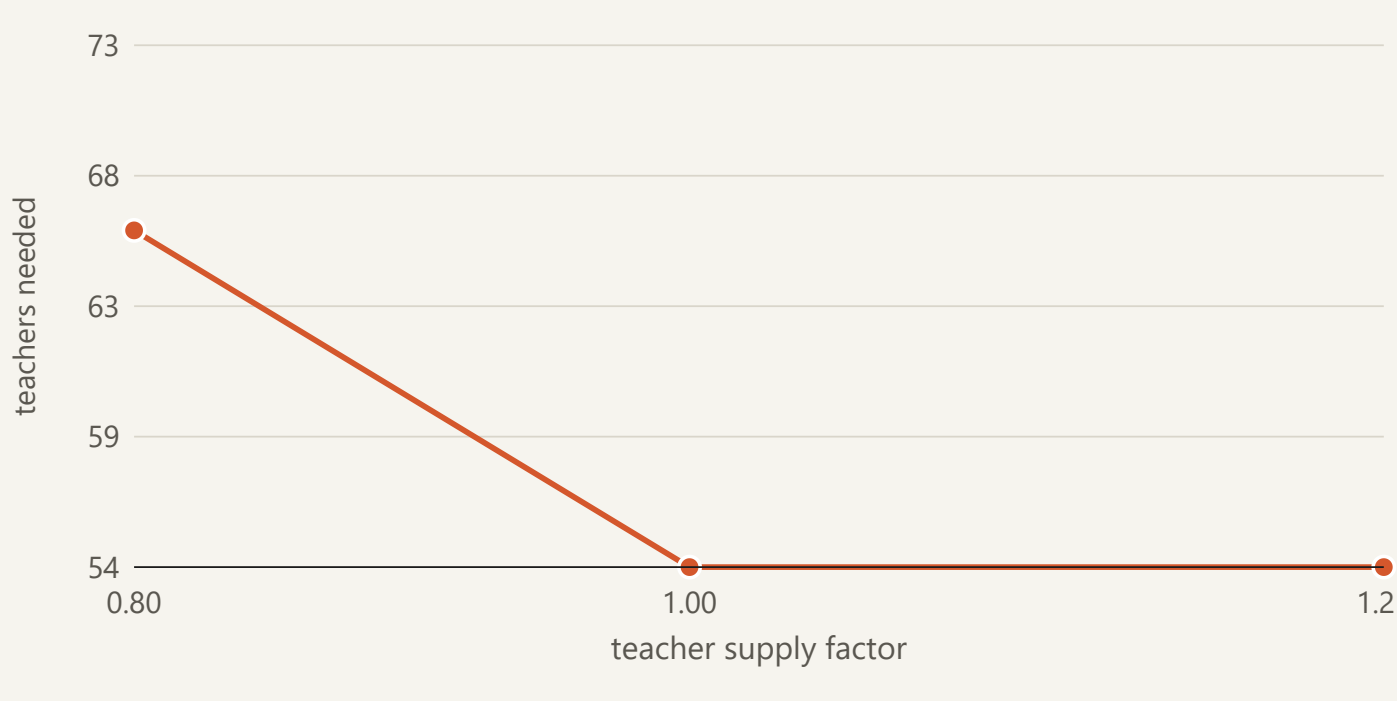


Fig. 22 — *Staffing is an output: the number of teachers needed as supply tightens.*

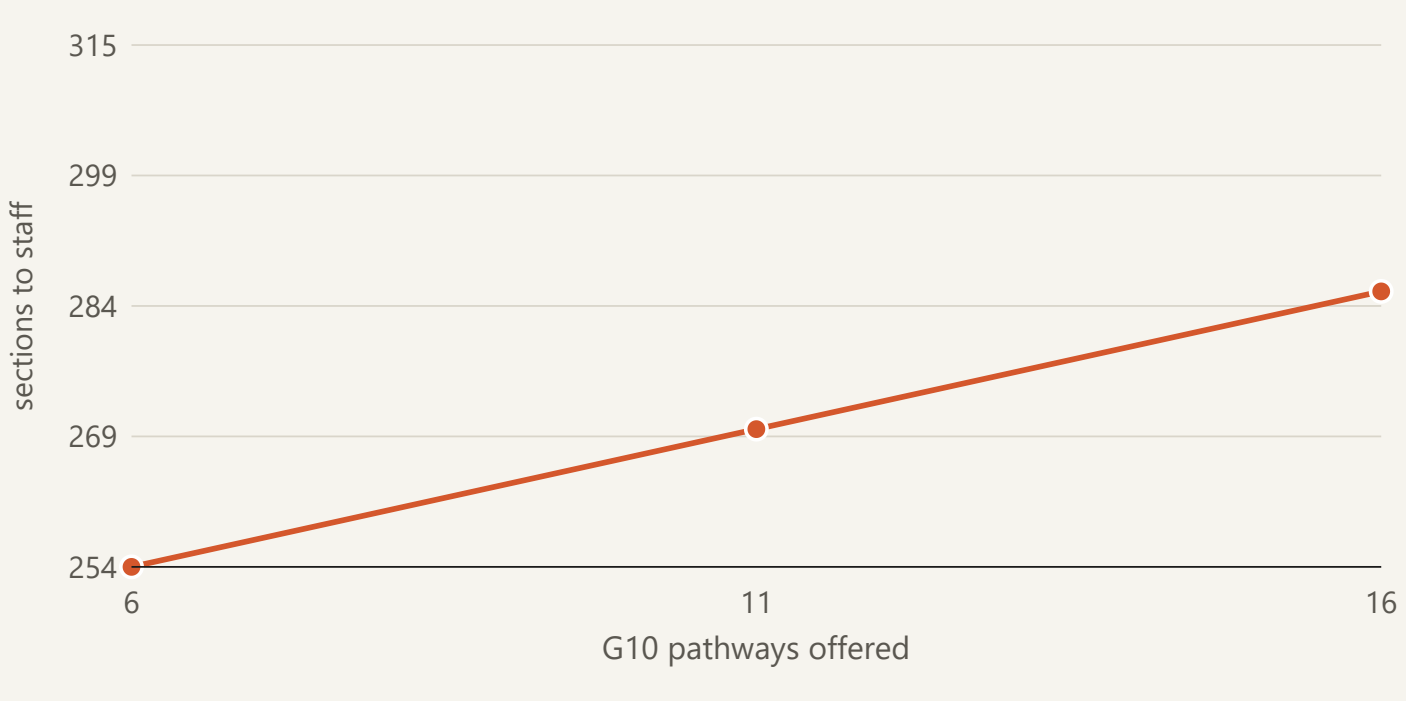


Fig. 23 — *The price of choice: more pathways means smaller classes and more sections.*

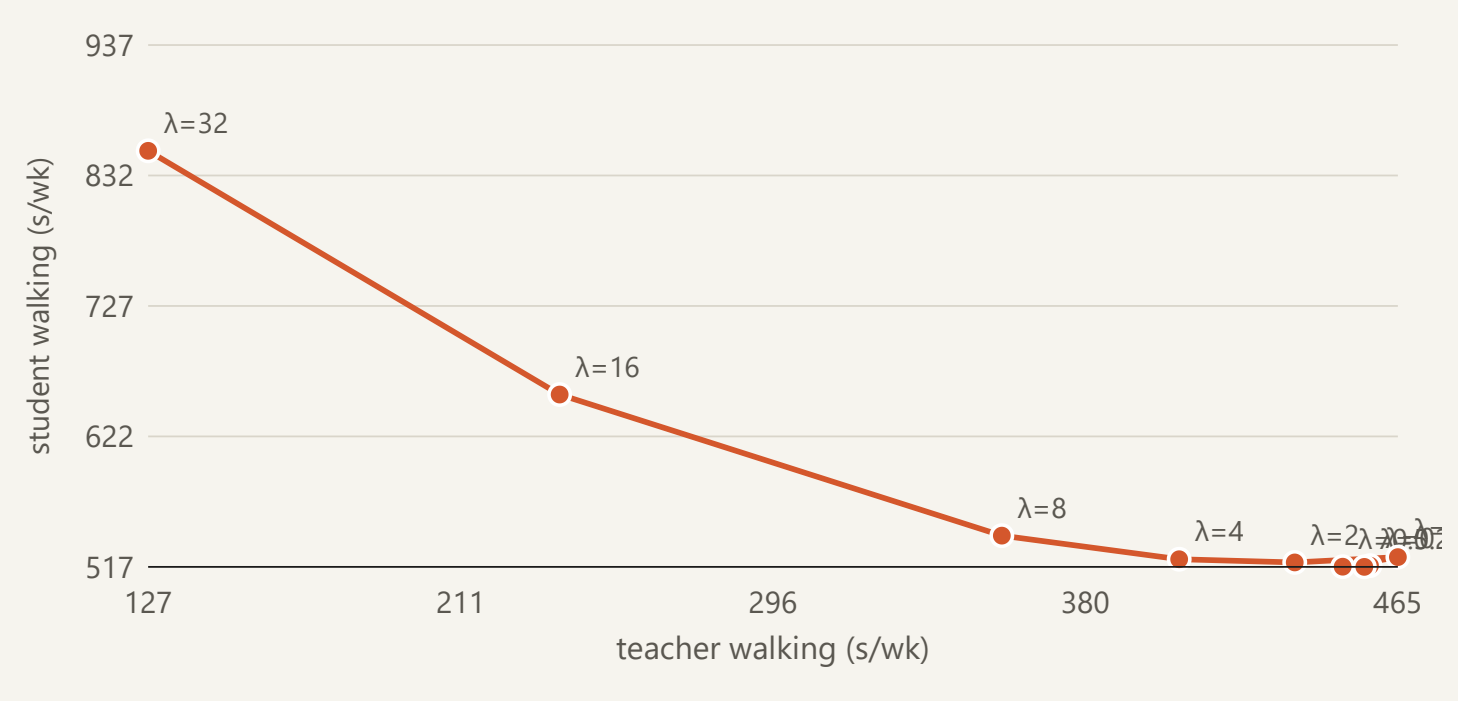


Fig. 24 — *Pareto front: moving walking from many students to few teachers is cheap, until λ becomes large.*

G When circumstances change

Q6 — repair ladder: substitute → re-room → re-time only what must move

Room B217 closed for the week (B2)

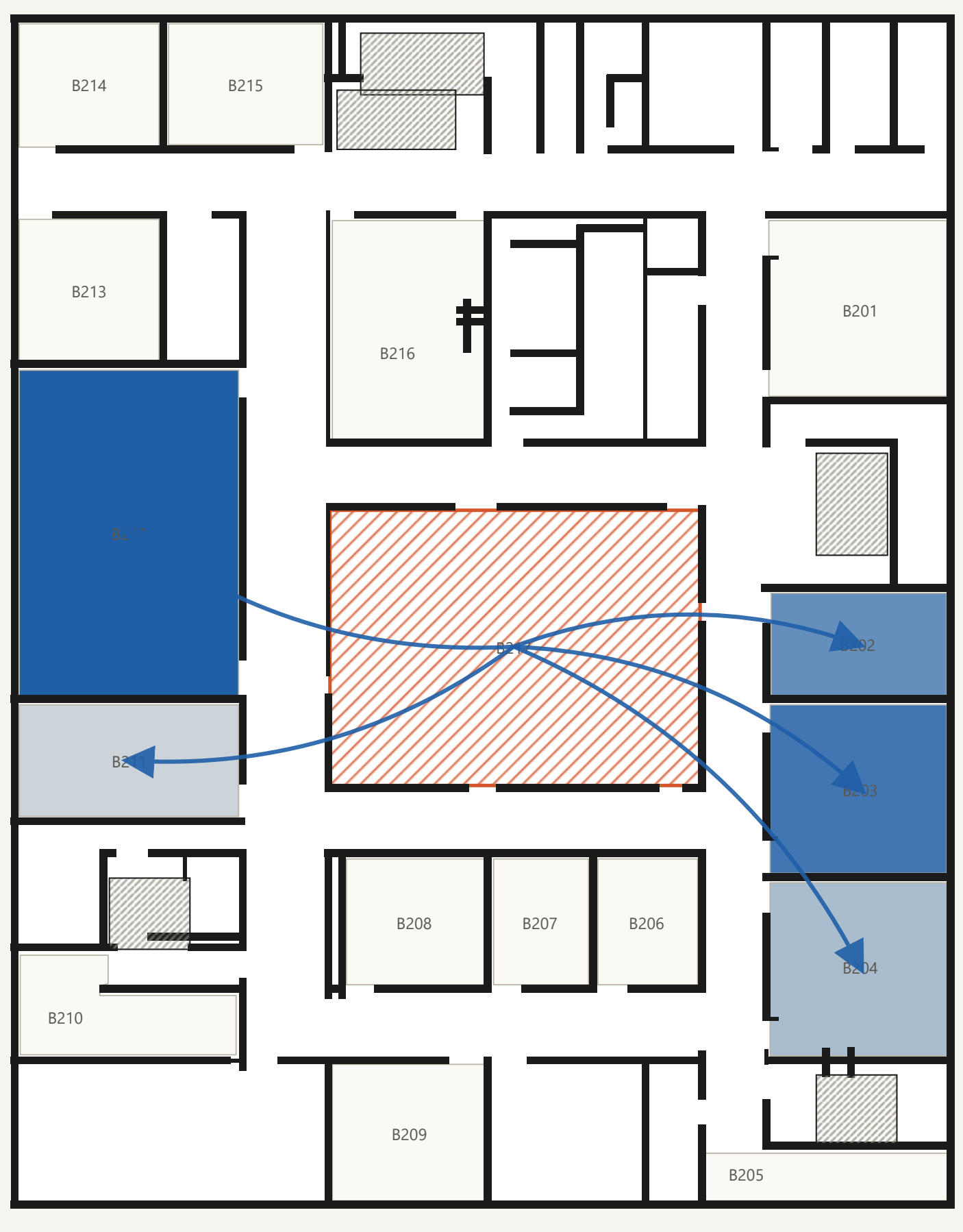


Fig. 25 — *28 lessons lose their room; 28 are re-roomed in a free room of the same kind,* **0 lessons change time**. *Blue = receiving rooms.*

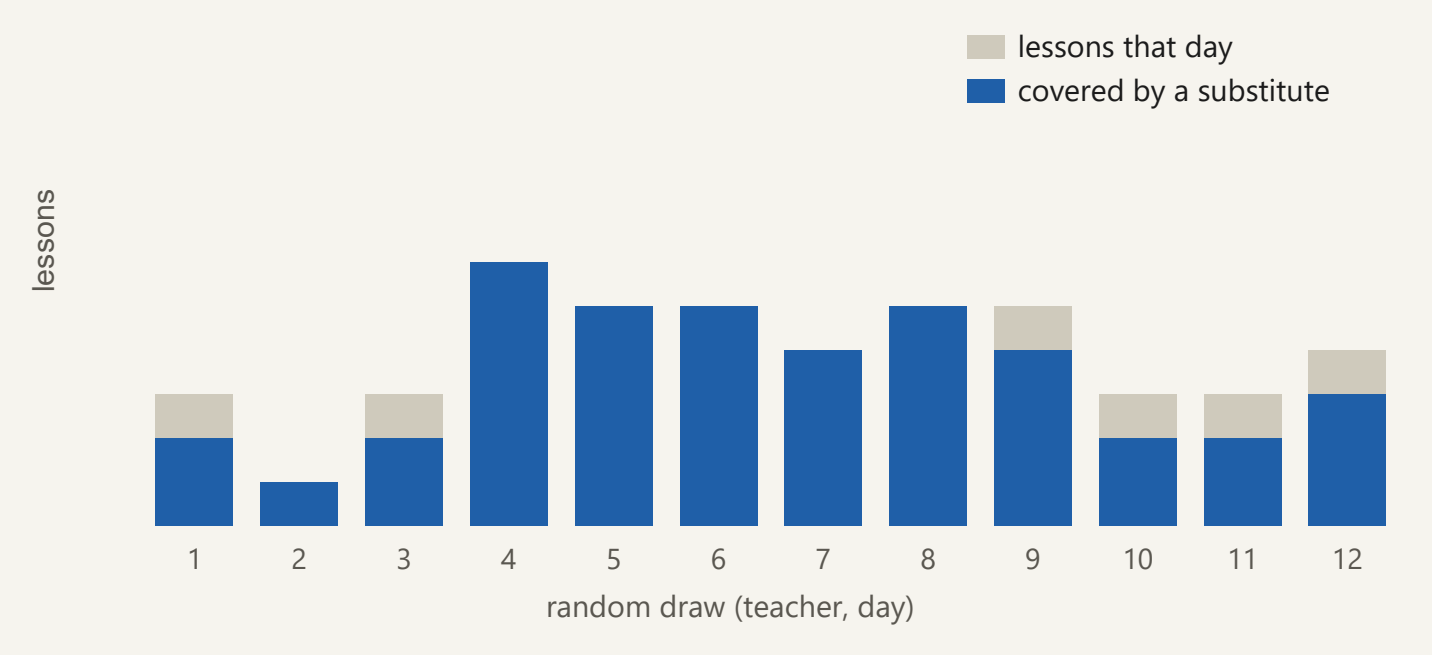


Fig. 26 — *Teacher absent for a day (p = 0.4): a qualified, free colleague covers 87% of lessons; nothing in the grid moves.*

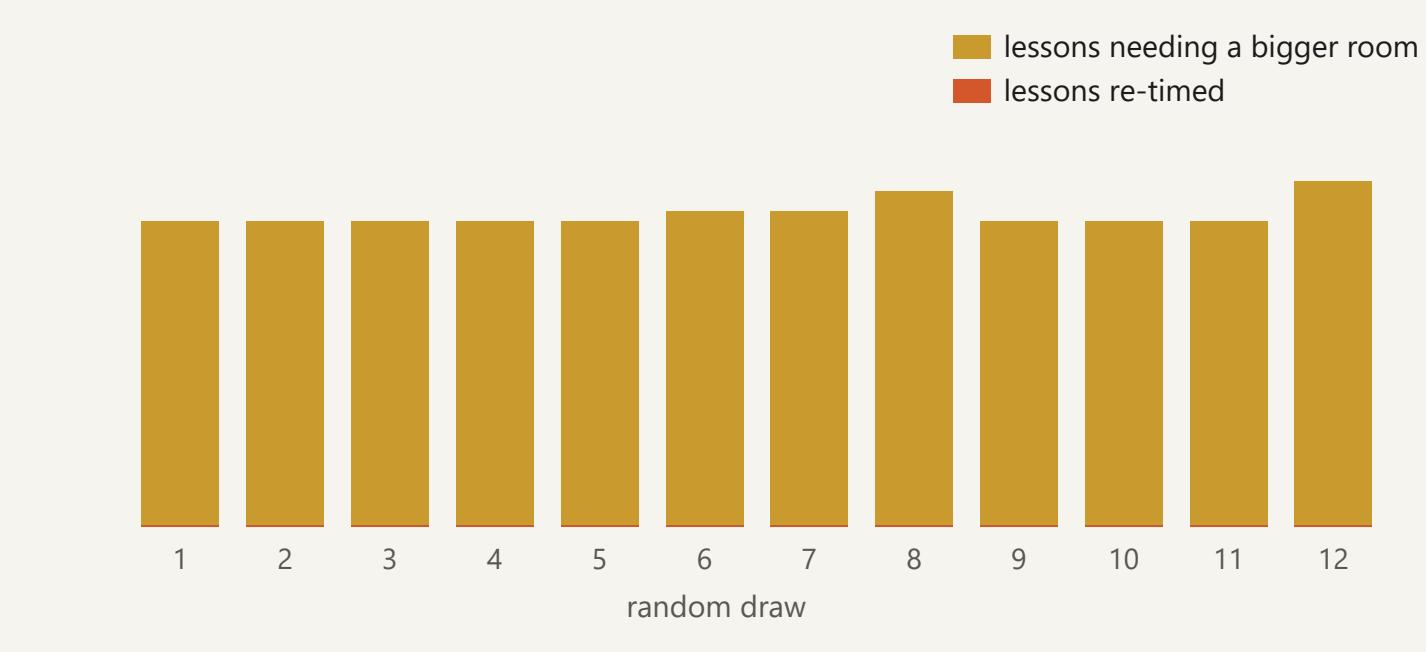


Fig. 27 — *Enrollment +5% (p = 0.3): rooms that no longer fit are swapped for bigger ones; the grid itself keeps its times.*

Expected cost over the three scenarios: **E[moved lessons] = 0.00**, E[room changes] = 14.4, infeasible rate 0%, over 12 draws each. Every repaired grid is run back through the independent checker.

H Limits, and how to reproduce

what we did not prove

- **One real week only.** The current system is an emulator calibrated to it (about 20 vs 21 changes); everything else is simulated. Registrar CSVs drop in.
- **Operating envelope.** Valid grids were found at k ≈ 0.87 with up to 20% of rooms removed; at k ≥ 0.95 or ≥ 40% rooms removed none was found in the time limit (Fig. 20).
- **Not proven optimal.** CP-SAT stopped at its time limit, 78% from its bound: "optimized" means best feasible found.
- **Pathway classes trade choice for feasibility** (Fig. 4); the price is quantified in F.
- **Minutes:** sections of one course can still differ by up to 36% in weekly minutes; the next rule to add is equality across sections.
- **Reproduce:** `python -m tt.run`, `python -m tt.sensitivity`, `python -m tt.poster2`.
- Every number on this poster is read from `output/*.json`; seeds and the config hash are fixed.

Constraints from the course catalog · building: 6 floors, 103 rooms, stair-aware walking times · simulated school, seed 20261024

Python, OR-Tools CP-SAT